UNIFAAT 2026.2

07 - Views e Elasticsearch

Prof. Luan Lourenço

Estático x Dinâmico

```
# estático · Nginx (Aula 01)
→ arquivo pronto no disco
→ igual para todo mundo
→ nenhum código roda no servidor backend
```

```
# dinâmico · Node
→ o servidor executa código
→ DW: JSON montado do banco
→ muda a cada requisição
```

```
# o HTML também é só texto: mesmo corpo "<h1>Olá</h1>", dois headers
Content-Type: text/plain  → o navegador mostra os caracteres <h1>Olá</h1>
Content-Type: text/html   → o navegador monta a página e cria o DOM
```

Qualquer arquivo pode sair do backend. Quem decide o que fazer com os bytes é o Content-Type.

O controller monta o HTML?

```
TasksPageController
  → busca as tarefas do usuário no banco
  → concatena o HTML numa template string: `<li>${task.name}</li>`
  → responde com Content-Type: text/html
```

- → **Mistura responsabilidades**: lógica HTTP e formatação do HTML no mesmo arquivo
- → **Não escala**: cabeçalho, menu e rodapé copiados em todo controller
- → **Não escapa**: uma tarefa chamada `<script>` executa no navegador (XSS, Aula 08)

Funciona, mas é má prática.

MVC: o V que faltava

- → **Model**: acessa os dados (já existia)
- → **Controller**: trata a requisição e chama os Models (já existia)
- → **View**: recebe dados prontos e devolve HTML (a novidade)

```
GET /tasks
  ↓ middlewares (cookie, auth)
  ↓ controller
  ↓ models (Postgres)
  ↓ view (HTML com os dados)
  ↓ Content-Type: text/html
```

O HTML passa pelo ciclo de vida HTTP inteiro, como o JSON da API sempre passou: middlewares, banco e regras de negócio valem para a página.

Login: quem decide a sessão?

```
# antes · login.html estático
Nginx entrega login.html a todos
  ↓ a página chega, o JS roda
  ↓ checkAuthentication()
  ↓ GET /users/me
  ↓ token ok? redireciona
decide DEPOIS da página chegar
```

```
# agora · View + middleware
GET /login
  ↓ ParseCookiesMiddleware
  ↓ RedirectIfAuthenticated...
     token ok → 302 /tasks.html
  ↓ LoginViewController → login.ejs
decide ANTES do HTML existir
```

O Nginx não sabe o que é um JWT; um middleware sabe. O inverso protege página privada: sem token, redirect para /login. (código completo: raio-x da aula)

EJS: Embedded JavaScript

Um arquivo que é quase HTML, com marcações onde entram os dados. Instalado com npm, configurado no _web.js (view engine ejs, views em resources/views).

```
<%= valor %>    imprime ESCAPADO: um <script> vira texto
<%- valor %>    imprime sem escape: só HTML confiável
<% código %>    executa JS sem imprimir: if, laços
<%# nota %>     comentário: não vai para o HTML
```

docker O node_modules fica num volume nomeado, criado uma vez só. Lib nova: `down`, `docker volume rm fe-2026_nodemodules-volume`, `up -d --build`.

Hello World: a primeira view

```
GET /hello?nome=Luan
  ↓ controller: só decide os dados (nome, padrão "mundo"; hora do servidor)
  ↓ hello.ejs: Olá, <%= nome %>!
  ↓ "Olá, Luan!": um HTML que não existe em disco
```

```
# ?nome=Luan
Olá, Luan!
```

```
# ?nome=<script>alert(1)</script>
vira texto na tela, nada executa
```

O <%= escapa por padrão. Escape e XSS são assunto da Aula 08.

Elasticsearch: busca

- Filtro por N campos em SQL: `ILIKE '%x%'` lê linha a linha
- Um índice por combinação de filtros? Impossível
- Sem relevância e sem linguagem: relatório e relatorio são textos diferentes

```
índice    ≈ tabela
documento ≈ linha, em JSON
_id       = PK do Postgres
# índice invertido
relatorio → tarefas 42, 57, 90
```

```
# busca vetorial, em uma linha
texto → vetor de significado: "pagar boleto" acha "quitar conta de luz"
```

Buscar = ler listas prontas e cruzá-las, com nota de relevância (score).

importante

Sem porta exposta

```
navegador ──✗──→ elasticsearch_host:9200   (nenhuma porta no host)
navegador → Nginx → servidor web ──→ elasticsearch_host:9200  ✓ busca
                         worker  ──→ elasticsearch_host:9200  ✓ alimenta
```

- Mais um container (`elasticsearch:9.5.5`), **sem bloco ports**: só a rede interna (app-network) alcança
- Falam com ele o **servidor web** e o **worker**; o navegador nunca, toda busca passa pela API
- Sem usuário e senha só em ambiente local, e só porque não há porta exposta

---

Teste de dentro da rede: `docker compose exec nodeweb-container curl http://elasticsearch_host:9200`

o ponto mais importante

Busca em duas etapas

```
1) Elasticsearch: a query complexa (texto + N filtros)
     ← só os ids, já por relevância: 57, 42, 90
2) Postgres: WHERE id IN (57, 42, 90), pela chave primária
     ← os dados completos, reordenados pelos ids
```

→ O Elasticsearch decide **QUEM** aparece. O Postgres decide **O QUE** aparece.

Excluída no Postgres? Não aparece. Alterada? Vem a versão atual. O Postgres segue sendo a fonte da verdade.

Exemplo: a busca

Usuário 7 busca **"relatorio"**, só **pendentes**, criadas **desde 01/10**.

```
# índice tasks no Elasticsearch (cópia para busca)
_id  name                  is_done  id_user  created_at
42   Entregar relatório    false    7        03/10  ✓
57   Relatórios do mês     false    7        05/10  ✓
90   Revisar relatorio     false    7        02/10  ✓
61   Relatório antigo      true     7        03/10  ✗ já concluída
73   Relatório da Ana      false    3        04/10  ✗ outro usuário
77   Comprar pão           false    7        06/10  ✗ texto não bate
```

→ "relatorio" acha relatório, Relatórios e relatorio: com o analisador de português no mapping, o Elastic **normaliza** acento, maiúscula e plural. Resposta: só os ids, por relevância: **57, 42, 90**.

Exemplo: a volta ao Postgres

```
# 200 ms antes, no Postgres: tarefa 90 excluída, tarefa 42 renomeada
# o worker ainda não sincronizou o Elastic
Elastic devolveu  → 57, 42, 90
Postgres          → WHERE id IN (57, 42, 90)
57  Relatórios do mês                 ✓
42  Entregar relatório final          ✓ nome novo, direto da fonte da verdade
90  (não existe mais: o WHERE IN não acha)  ✗ some do resultado
```

> **!** O Elastic estava atrasado e a resposta saiu **certa** mesmo assim: quem aparece veio do Elastic, o que aparece veio do Postgres.

O WHERE IN não garante a ordem: reordena pelos ids do Elastic.

Alimentação assíncrona

Indexar não é o que o usuário pediu: sai do ciclo HTTP.

- → **RabbitMQ** (conhecido): a fila
- → **Worker** (conhecido): processo Node sem rota HTTP
- → **Elasticsearch** (novo): o banco de busca

```
POST /me/tasks
  ↓ grava no Postgres
  ↓ job com só o id na fila
  ↓ responde 201
··· outro container ···
worker lê o estado atual
  ↓ upsert ou DELETE no Elastic
```

API REST do Elastic: PUT (upsert) · _update · DELETE · _search · _bulk (carga inicial).

Exemplo: a vida da tarefa 95

```
# cria
t = 0 ms    POST /me/tasks → INSERT no Postgres (id 95)
t = 3 ms    job na fila: { job: "SyncTaskSearchJob", payload: { id: 95 } }
t = 5 ms    201 para o usuário (não esperou o Elastic)
t = 40 ms   worker: PUT /tasks/_doc/95 → "result": "created"
t ≤ 1 s     refresh do índice: a 95 aparece na busca
# edita    mesmo job, mesmo id → PUT de novo → "updated"
# exclui   mesmo job: a 95 não existe no Postgres → DELETE → "deleted"
```

→ Um job só, que recebe **só o id** e copia o estado atual do Postgres: inserir, editar e excluir viram a mesma operação de sincronizar.

Até o refresh, a busca não vê a 95. Dado errado, nunca.

importante

Consistência eventual

Fila + refresh do índice (~1 s): por alguns milissegundos, Postgres e Elasticsearch estão diferentes.

```
# o preço: busca e agregação
contagem uma unidade a mais/menos
concluída ainda nas pendentes
recém-criada ainda não aparece
```

```
# a garantia: WHERE IN no Postgres
nunca um dado errado
nunca um registro que não existe
a integridade fica no Postgres
```

A busca atrasa milissegundos; o dado exibido nunca fica errado. (código completo: raio-x da aula)

TF 07

Views e Elasticsearch

Na tela de tarefas, um modal para criar uma tarefa e atribuí-la a outro usuário, encontrado pela API de search (qualquer campo).

https://docs.google.com/forms/d/e/1FAIpQLSdJEQMCfZT5hO48KjOPJpwLvxU4O7_JJaMZPW3w5OmqTCxZUQ/viewform?usp=publish-editor

---

Prazo: 26/10/2026 às 23:59:59 · detalhes completos em TF07.md.